

VERBATIM: A Robust, Hallucination-Resistant Meeting Secretary Pipeline with Verifiable Evidence Anchoring and Deterministic Guardrails

Technical Architecture, Design Rationale, End-to-End Pipeline, and Empirical Validation

Machine Learning Systems Research Group | Inter IIT Tech Bootcamp, ML Track

Date of Publication: October 2026 | Document Identifier: VERBATIM-TR-2026-01

Abstract

Automated transcription and summarization of spoken engineering meetings remains notoriously susceptible to acoustic mishearings, hallucinated consensus, phantom action item attribution, and arbitrary temporal extrapolation. Typical naive pipelines—cascading an Automatic Speech Recognition (ASR) engine into a single-turn Large Language Model (LLM) prompt—fail consistently in real-world scenarios due to unconstrained generative drift and the lack of auditable evidence links. In this technical report, we present **Verbatim**, a resilient, local-first meeting secretary system engineered around three architectural pillars: (1) *Evidence Follows You*, guaranteeing bidirectional cross-referencing between synthesized minutes and exact sub-second audio waveforms; (2) *Progressive Canvas Architecture*, eliminating opaque loading spinners via Server-Sent Events (SSE) streaming; and (3) *Honest by Construction*, enforcing deterministic Python guardrails to govern probabilistic LLM outputs.

Verbatim introduces a dual-model decoupled architecture separating transcript repair (Qwen) from structured documentation (Gemma), bounded by seven mathematical and lexical invariance rules (invariance of numbers, negations, and modal commitment verbs). Furthermore, Verbatim deploys a five-stage decomposed secretary workflow incorporating localized lexical support scoring (≥ 0.40), generic pronoun demotion, verbatim deadline preservation, and automated bidirectional Markdown-to-JSON structural parity validation. On empirical benchmarks comprising complex, multi-speaker technical deliberations featuring phonetic traps, deferred decisions, and ambiguous ownership, Verbatim demonstrates a 100% ground-truth verification pass rate across all decision and task constraints, zero false-consensus hallucinations, and full hardware adaptability across consumer GPU and CPU environments.

Keywords: Automated Speech Recognition, Large Language Models, Meeting Summarization, Hallucination Mitigation, Deterministic Guardrails, Audio-Text Evidence Grounding, Local-First Systems.

1 Problem Statement and Motivation

Spoken technical meetings are the primary locus of organizational decision-making, architectural consensus, and task delegation in modern engineering teams. Despite the pervasive adoption of audio and video conferencing tools, turning recorded spoken dialogue into an accurate, unambiguous, and accountable written record remains an unresolved operational bottleneck.

Human professionals cannot afford to re-listen to hours of raw recording to confirm whether an engineering decision was finalized or merely proposed. Conversely, off-the-shelf automated meeting assistants and naive AI transcription pipelines exhibit severe failure modes that actively degrade team trust. When an automated tool attributes an unassigned task to an arbitrary attendee, invents an artificial deadline, or misinterprets a casual brainstorm as an approved architectural mandate, the operational risk to the engineering organization is catastrophic.

1.1 Vulnerabilities of the Naive Pipeline

The standard paradigm deployed in commercial and hackathon-grade meeting assistants is depicted below:

$$\text{Raw Audio} \xrightarrow{\text{ASR}} \text{Raw Transcript} \xrightarrow{\text{Single LLM Call}} \text{Markdown Minutes} \quad (1)$$

While architecturally simplistic, this naive cascade suffers from five fundamental, recurring vulnerabilities:

- (a) **Acoustic-Phonetic Degradation of Domain Jargon:** Standard speech-to-text models (such as Whisper) trained on broad web-scale corpora frequently fail on specialized technical vocabulary. In software engineering dialogues, terms like *Memcached*, *PostgreSQL*, *OAuth scopes*, and *gRPC* are routinely misheard as phonetically proximate everyday phrases: “*Memcatch*”, “*post-GIR SQL*”, “*OOFscopes*”, and “*GRPC*”. Without targeted contextual repair, these phonetic errors propagate directly into the executive documentation.
- (b) **Semantic Drift and False Consensus (“Politeness Hallucination”):** Large Language Models are biased toward producing cohesive, definitive narratives. When presented with conversational debate—such as an engineer proposing a migration to gRPC, followed by a colleague raising latency concerns and the team

lead deferring the discussion to the next sprint—naive LLM summarizers routinely classify the proposal as an agreed, finalized decision. Generative summarizers conflate *deliberation* with *consensus*.

- (c) **Phantom Accountability and Pronoun Misattribution:** Spoken conversations rely heavily on collective, non-specific pronouns: “*We should rotate credentials*”, “*Someone needs to inspect the Kafka lag*”, or “*The team will look into this*”. Probabilistic LLMs prompted to extract action items with fields ⟨Task, Owner, Deadline⟩ exhibit strong tendencies to fabricate accountability—either arbitrarily binding the task to whichever speaker spoke last, or converting collective pronouns like “*we*” into individual team members.
- (d) **Temporal Hallucination and Unwarranted Date Extrapolation:** Spoken deadlines are inherently relative and contextual (e.g., “*by Thursday*”, “*before the audit on the 15th*”, or “*no date yet until we scope it*”). Standard LLM summarizers attempt to resolve these relative dates into absolute calendar dates (e.g., ISO dates like 2026-10-15T00:00:00Z) without knowing the meeting’s recording date, timezone, or year, resulting in silent and dangerous temporal errors.
- (e) **The Opaque “Black Box” Audit Deficit:** When a human reviewer is presented with a synthesized bullet point—such as “*Migrate session cache from Memcached to Redis*”—they have no straightforward mechanism to verify whether this reflects actual team consensus without manually scrubbing through an hour of audio. The absence of fine-grained, bidirectional linking between text items and audio timestamps renders automated summaries unverifiable.

1.2 Operational Hardware and Latency Constraints

In addition to accuracy vulnerabilities, practical deployment environments impose rigid constraints:

- **Constrained Hardware Footprint:** The system must execute locally on consumer-grade engineering hardware (such as laptops equipped with 6 GB to 8 GB VRAM, e.g., NVIDIA GeForce RTX 4060, or multi-core x86/ARM CPUs) without exceeding physical VRAM limits or inducing Out-Of-Memory (OOM) crashes.
- **Deterministic Reproducibility:** Output generation must be completely reproducible across multiple runs on identical input, enforcing temperature $T = 0$, fixed random seeds, and versioned prompts.
- **Responsive Interactive UX:** Reviewers cannot be subjected to multi-minute opaque spinner screens. The interface must provide progressive, visible workspace updates from the moment audio ingestion commences.

2 System Architecture and Design Philosophy

To systematically resolve the vulnerabilities detailed in Section 1, **Verbatim** is engineered around three foundational design principles that govern its architectural topology.

The Three Foundational Design Pillars of Verbatim

1. **Evidence Follows You:** Every decision, action item, minute point, and transcript correction is irrevocably anchored to sequential segment IDs. Clicking any item in the record view causes the transcript to scroll, applies an active visual marker, displays the precise bounding region on the audio waveform, and immediately plays the underlying audio range with a calibrated lead-in and lead-out buffer.
2. **The Workspace is the Progress Screen:** Eliminating blocking splash pages. From the instant media is accepted, a two-pane interactive workspace is rendered. Raw ASR segments stream in as they are generated; phonetic corrections appear as blue insertions as the refiner identifies them; and meeting minutes, decisions, and tasks populate section-by-section via Server-Sent Events (SSE).
3. **Honest by Construction:** Probabilistic LLMs are never permitted to apply destructive full-text mutations or fabricate metadata unchecked. The Refiner can only propose localized find/replace spans, validated by deterministic Python guardrails. Missing owners or deadlines are explicitly rendered as an **Unspecified** pill, and unsettled discussions are isolated into an explicit **Unresolved** register.

2.1 Decoupled Dual-LLM Family Architecture

A critical design vulnerability of single-model meeting systems is *self-reinforcing cognitive bias*: if a single LLM model family both repairs the transcript and documents the meeting, it consistently overlooks its own interpretive distortions. Verbatim explicitly decouples the cognitive workload across two distinct model families:

- **The Refiner (LLM #1):** Dedicated exclusively to acoustic and technical vocabulary repair. Configured by default to the **Qwen** family (e.g., **qwen3:8b** or **qwen2.5:7b**), with reasoning/thinking disabled to maximize latency and enforce strict structural JSON output.
- **The Documenter (LLM #2):** Dedicated to hierarchical synthesis, topic grouping, decision extraction, and task assignment. Configured by default to the **Gemma** family (e.g., **gemma3:12b** or **gemma3:4b**).

By utilizing different model architectures and pretraining corpora, the Documenter acts as an independent auditor of the transcript text, preventing intra-model confirmation loops.

Table 1: Systemic Comparison: Initial Prototype Baseline vs. Verbatim Production Architecture.

Subsystem	Baseline Prototype (SOLUTION.md)	Verbatim System Architecture
User Interface	Streamlit with tabs and static sidebars. Blocking spinner pages during execution.	FastAPI + Vite/React 19 + TypeScript + Zustand. 46/54 resizable split-pane with live SSE streaming.
ASR Stack	WhisperX + wav2vec2 alignment + pyannote diarization. Heavy gated dependencies.	faster-whisper (CTranslate2) with built-in Silero VAD and sub-word timestamp extraction. Zero gated tokens.
Transcript Refinement	LLM rewrites the complete 4000-word text; difflib attempts to infer diffs.	LLM outputs discrete atomic tuples <code>(id, orig, corr)</code> . Deterministic Python safety guardrails apply edits.
Documentation Calls	2 monolithic multi-field LLM calls followed by optional unstructured self-reflection.	5 decomposed sequential calls (Summary, Minutes, Decisions, Tasks, Verification) feeding progressive UI.
Tentative Actions	Ambiguous boolean <code>status</code> mixed into the primary action item list.	Explicit bifurcation into <code>ActionItem</code> (committed) vs. <code>Unresolved</code> (<code>possible_task</code>).
Evidence Grounding	Fuzzy text matching on quoted strings at an arbitrary threshold (≥ 80).	Segment-ID indexing + normalized lexical support scoring + speaker/deadline proximity windows.
Model Strategy	Assumes single high-end GPU with 32k context on an 8 GB card (leads to OOM).	Three hardware-adaptive profiles (<code>lite</code> , <code>standard</code> , <code>quality</code>) with CPU fallback and dynamic VRAM caching.

2.2 Architectural Paradigm Shift: Baseline vs. Verbatim

Table 1 illustrates the systemic architectural improvements implemented in Verbatim compared to initial baseline architectures (e.g., SOLUTION.md specifications).

2.3 Hardware-Aware Profile Matrix

To guarantee that Verbatim executes reliably on diverse reviewer environments, the system implements dynamic hardware profiling as specified in `config.yaml` and managed by `backend/verbatim/gpu.py`. Table 2 defines the operational profiles.

Table 2: Hardware-Adaptive Profiles and Resource Allocations in Verbatim.

Parameter	Lite Profile	Standard Profile	Quality Profile
Target Hardware	≤ 6 GB VRAM or CPU	8 GB VRAM (e.g. RTX 4060)	≥ 12 GB VRAM (e.g. RTX 4080)
STT Engine	<code>distil-large-v3</code> (int8)	<code>large-v3</code> (float16)	<code>large-v3</code> (float16)
Refiner LLM	<code>qwen3:4b</code> / <code>qwen2.5:3b</code>	<code>qwen3:8b</code> / <code>qwen2.5:7b</code>	<code>qwen2.5:14b</code>
Documenter LLM	<code>gemma3:4b</code>	<code>gemma3:12b</code>	<code>gemma3:27b</code> / <code>qwen2.5:32b</code>
Refiner Context	8,192 tokens	12,288 tokens	16,384 tokens
Doc Context	12,288 tokens	16,384 tokens	32,768 tokens
VRAM Unload	Forced after stage	Forced after stage	Keep in memory
Max Duration	30 minutes	60 minutes	120 minutes

2.4 Concurrency Control and Persistence Model

Because local consumer GPUs represent a non-shareable physical resource, running concurrent LLM and Whisper inferences causes immediate out-of-memory crashes or thrashing. Verbatim enforces a strict single-worker execution queue managed by `backend/verbatim/jobs.py` using a `ThreadPoolExecutor(max_workers=1)`.

Run state is persisted atomically to disk under isolated run directories:

`runs/{run_id}/ = {meta.json, audio.wav, peaks.json, raw.json, refined.json, meeting_record.json, events.jsonl}` (2)

Every state mutation emits an immutable, append-only log entry into `events.jsonl`. If the client browser disconnects or the user refreshes the page mid-execution, the frontend reconnects via SSE with `Last-Event-ID: N`, sequentially replaying the event log through a pure Zustand reducer to reconstruct exact application state without restarting backend computation.

3 Ingestion and Signal Preprocessing

Audio ingestion, implemented in `backend/verbatim/ingest.py`, establishes the acoustic foundation for the entire downstream pipeline. Meeting recordings arrive in arbitrary container formats, sample rates, channel configurations, and compression standards.

3.1 Demuxing, Format Normalization, and Silence Preflight

Using PyAV (direct FFMpeg bindings), Verbatim accepts a wide whitelist of extensions (`.wav`, `.mp3`, `.m4a`, `.aac`, `.flac`, `.ogg`, `.opus`, `.webm`, `.mp4`, `.mov`, `.mkv`) up to a 500 MB ceiling. The audio stream is extracted, decoded, converted to mono, and resampled to a standardized **16 kHz, 16-bit linear PCM WAV** container.

Before committing expensive GPU resources, the audio undergoes deterministic preflight validation:

1. **Duration Boundaries:** $T_{\text{audio}} \geq 2.0$ seconds. Sub-2-second files are rejected immediately with `EMPTY_FILE` or `INVALID_AUDIO`.
2. **Silence Rejection via RMS dBFS:** Entire-file Root Mean Square (RMS) energy is calculated:

$$x_{\text{rms}} = \sqrt{\frac{1}{N} \sum_{i=1}^N x[i]^2}, \quad \text{dBFS} = 20 \log_{10} \left(\frac{x_{\text{rms}}}{x_{\text{max}}} \right) \quad (3)$$

where $x_{\text{max}} = 32768$ for 16-bit integer PCM. If $\text{dBFS} < -55.0$, the file is identified as silent or muted, returning an immediate, actionable HTTP 400 error (`SILENT_AUDIO`).

3.2 Downsampled Waveform Peak Synthesis

To achieve instantaneous client-side audio rendering without requiring the browser to download and decode a 100 MB audio buffer into Web Audio memory, Verbatim pre-computes an array of **1600 downsampled peak values** during the ingestion phase.

The audio signal is partitioned into 1600 uniform temporal bins. For each bin b_k , the absolute maximum peak is computed:

$$p_k = \max_{j \in \text{bin}_k} \frac{|x[j]|}{32768.0}, \quad k \in [0, 1599] \quad (4)$$

The resulting array is persisted as `peaks.json`. The frontend feeds these normalized floats directly into WaveSurfer.js v8, rendering pixel-perfect audio waveforms instantly across all viewport dimensions.

4 Speech-to-Text Transcription Subsystem

Transcription, implemented in `backend/verbatim/stt.py`, converts normalized PCM audio into an ordered sequence of discrete, timestamped transcript segments:

$$\mathcal{S} = \{s_1, s_2, \dots, s_M\}, \quad s_i = \langle \text{id}_i, t_{\text{start},i}, t_{\text{end},i}, \text{text}_i, \mathcal{W}_i \rangle \quad (5)$$

where $\mathcal{W}_i$ represents word-level timestamps $\langle \text{word}, t_{\text{start}}, t_{\text{end}}, \text{prob} \rangle$.

4.1 Engine Selection: faster-whisper over WhisperX

While WhisperX offers phoneme-level forced alignment via `wav2vec2` and speaker diarization via `pyannote-audio`, it imposes brittle dependency pins (specific PyTorch versions, Hugging Face auth tokens for gated `pyannote` models). A single network failure or token misconfiguration renders WhisperX un-runnable on external evaluation machines.

Verbatim utilizes **faster-whisper**, a CTranslate2 re-implementation of OpenAI’s Whisper model. **faster-whisper** provides:

- 4× faster transcription speed and 50% lower VRAM consumption compared to vanilla PyTorch.
- Integrated Silero Voice Activity Detection (VAD) with `min_silence = 500` ms, preventing Whisper from hallucinating on ambient background noise.
- Integrated sub-word and word-level alignment without external neural alignments.

4.2 Acoustic Bias Priming and Hallucination Suppression

Whisper models frequently suffer from acoustic hallucination loops (e.g., repeating the same sentence 30 times during audio fade-out) and temperature degradation. Verbatim applies four hardening techniques:

1. **Initial Prompt Priming:** Before transcription, Verbatim constructs a deterministic initial prompt containing meeting context, punctuation cues, and user-provided hotwords:

$$\mathcal{P}_{\text{init}} = \text{“Meeting discussion. Agenda: session cache, gRPC, PostgreSQL, OAuth, Terraform, Kafka, Grafana.”} \quad (6)$$

This biases beam search decoding toward correct spellings of technical terms without mutating model weights.

2. **Beam Search Configuration:** Fixed beam size of 5 with `best_of = 5`, disabling stochastic temperature fallback (temperature = 0.0) to guarantee deterministic output.
3. **Repetition Run Collapse:** If Whisper produces repetitive token sequences ($N \geq 3$ identical adjacent phrases), a post-ASR regex filter collapses the repetition into a single phrase, logging an acoustic warning.
4. **Lazy Generator Streaming:** Segments are yielded lazily as CTranslate2 finishes decoding them, broadcasting `transcript.segment` SSE events directly to the browser.

5 Transcript Refinement and Deterministic Guardrails

No automated ASR model, regardless of size, achieves zero Word Error Rate (WER) on fast, colloquial technical conversations. In spoken English, phonetic similarities routinely produce domain-specific corruptions. However, asking an LLM to “*rewrite the transcript to fix errors*” is catastrophic: generative models hallucinate dialogue, drop subtle sentences, and summarize text instead of correcting words.

Verbatim introduces the **Discrete Proposal Paradigm**: the LLM is restricted to proposing isolated, localized replacement spans, which are subsequently validated by a battery of seven deterministic Python safety guardrails before application.

5.1 Acoustic Contextual Hinting

Implemented in `backend/verbatim/refine/hints.py`, the system scans raw transcript tokens against a known domain vocabulary using RapidFuzz phonetic and token matching. Candidate corrections achieving a match score ≥ 72 (e.g., “*Memcatch*” $\rightarrow$ “*Memcached*”) are injected into the Refiner LLM’s system prompt as high-probability phonetic suggestions.

5.2 Chunked Sliding-Window Prompting

To prevent LLM context exhaustion and ensure strict adherence to JSON schemas, `backend/verbatim/refine/stage.py` processes transcripts in sliding windows of **40 editable segments**, flanked on either side by **4 read-only context segments**:

$$\mathcal{W}_{\text{window}} = \underbrace{[s_{k-4}, \dots, s_{k-1}]}_{\text{Read-Only Context}} \cup \underbrace{[s_k, \dots, s_{k+39}]}_{\text{Editable Window}} \cup \underbrace{[s_{k+40}, \dots, s_{k+43}]}_{\text{Read-Only Context}} \quad (7)$$

The Refiner is instructed to output an array of structured proposals conforming to Pydantic schema:

$$\mathcal{R}_{\text{proposal}} = \langle \text{segment_id}, \text{original}, \text{corrected}, \text{reason} \rangle \quad (8)$$

5.3 The Seven Deterministic Safety Guardrails

Every proposal emitted by the Refiner must pass all seven invariant rules codified in `backend/verbatim/refine/guardrails.py`. If a proposal violates even a single rule, it is permanently rejected, and the rejection reason is logged to `corrections.csv` for human audit. Table 3 specifies the mathematical formulation of each rule.

5.4 Multi-Span Replacement Algorithm

When multiple non-overlapping corrections target the same segment, naive string replacement causes character offset corruption. Implemented in `backend/verbatim/refine/apply.py`, Verbatim performs exact regex match indexing, sorts validated replacements in reverse order of character start offset (j_{start} descending), and splices the corrected spans into the segment string:

$$\text{text}_{\text{refined}} = \text{text}[: j_{\text{start}}] + \text{corrected} + \text{text}[j_{\text{end}} :] \quad (9)$$

Overlapping proposals targeting the same character span are detected, and only the highest-scoring non-overlapping proposal is accepted.

6 Structured Secretary Documentation Pipeline

Once the refined transcript is finalized, the **Documenter LLM** synthesizes the comprehensive meeting record. Rather than issuing a single, massive prompt attempting to generate minutes, decisions, and tasks simultaneously—which overloads the LLM’s attention mechanism—Verbatim decomposes the documentation process into **five sequential, specialized LLM invocations** orchestrated by `backend/verbatim/document/stage.py`.

6.1 Call-by-Call Breakdown

1. **Call 1 (Title, Summary, Attendees):** System prompt `doc_summary.v1.md`. Generates an executive distillation capturing meeting context. Attendees are extracted strictly from speech introductions and acknowledgments.
2. **Call 2 (Chronological Minutes):** System prompt `doc_minutes.v1.md`. Organizes the conversation into 3 to 7 structured topics. Every bullet point is required to cite its supporting `segment_ids`.

Table 3: The Seven Deterministic Safety Guardrails Governing Transcript Refinement.

Rule	Formal Criterion	Operational Purpose	Block Reason
Rule 1	$id \in \mathcal{S}_{\text{raw}} \wedge id \in \mathcal{S}_{\text{editable}}$	Prevents hallucinations outside the designated temporal window.	Referred to a line that doesn't exist
Rule 2	$\text{original} \subseteq \text{text}(\text{segment}_{id})$	Verifies that quoted text actually occurred verbatim in the recording.	Quoted text that isn't in the recording
Rule 3	$\text{orig} \neq \text{corr} \wedge \text{words}(\text{orig}) \leq 6$	Limits replacement to concise terminology; prevents wholesale rewrites.	Edit too large or identical
Rule 4	$\text{Multiset}(\mathcal{N}_{\text{orig}}) \equiv \text{Multiset}(\mathcal{N}_{\text{corr}})$	Forbids modifying numbers, digits, percentages, or spelled-out quantities.	Numbers, amounts, and dates must never change
Rule 5	$\text{Multiset}(\mathcal{M}_{\text{orig}}) \equiv \text{Multiset}(\mathcal{M}_{\text{corr}})$	Protects negation polarity (<i>not</i> , <i>never</i> , <i>none</i> , <i>neither</i> , <i>n't</i>).	Negations must never be added or removed
Rule 6	$\text{Multiset}(\mathcal{V}_{\text{orig}}) \equiv \text{Multiset}(\mathcal{V}_{\text{corr}})$	Protects modal commitment verbs (<i>will</i> , <i>won't</i> , <i>must</i> , <i>should</i> , <i>can</i> , <i>might</i>).	Commitment words must never change
Rule 7	$\mathcal{P}_{\text{names}} \cap \text{orig} \subseteq \text{corr}$	Ensures attendee names cannot be stripped or replaced with generic words.	Participant names must be preserved

- Call 3 (Decisions vs. Unresolved):** System prompt `doc_decisions.v1.md`. This call enforces the critical boundary between **agreed decisions** (unanimous or authoritative commitments) and **unresolved discussions**. Brainstorms, speculative ideas, and deferred items are routed to the **Unresolved** list under explicit categorical tags: **proposal**, **question**, or **deferred**.
- Call 4 (Action Items):** System prompt `doc_actions.v1.md`. Extracts committed tasks. The LLM is instructed: *"If an owner was not explicitly named, leave owner null. If a deadline was not spoken, leave deadline null. Do not fabricate."* Tentative suggestions without committed ownership are classified as **possible_task** and diverted to **unresolved**.
- Call 5 (Self-Check Verification Pass):** System prompt `doc_verify.v1.md`. If `config.document.verify_pass` is active, a dedicated verification prompt presents candidate decisions and tasks alongside the exact transcript text of cited segments ± 2 . The LLM classifies each candidate:
 - Decision verification: **confirmed_agreed** vs. **proposal_only** (demoted to **unresolved**) vs. **unsupported** (dropped).
 - Task verification: **confirmed_committed** vs. **tentative** (demoted to **possible_task**) vs. **unsupported** (dropped).

6.2 Ollama KV-Cache Reuse via Shared Prefix

All five prompt templates (`doc_*.v1.md`) share an identical leading system preamble (`prompts/doc_system.v1.md`) defining the meeting secretary persona, style guidelines, and JSON schemas. Because Ollama preserves the Key-Value (KV) cache for matching token prefixes across sequential requests, Calls 2 through 5 execute with near-zero prompt-ingestion latency, dramatically reducing total runtime.

7 Anti-Hallucination Grounding and Verification Engine

Even after five LLM calls, probabilistic models occasionally hallucinate subtle details. Verbatim places an absolute filter between the LLM output and the finalized record: the **Deterministic Grounding Engine** (`backend/verbatim/document/grounding.py`).

Decomposed Five-Stage Documentation Architecture

[Refined Transcript] $\rightarrow$

Call 1: Summary $\rightarrow$ Meeting Title (≤ 10 words) + Executive Summary (3–5 sentences) + Attendee Register

Call 2: Minutes $\rightarrow$ 3–7 Chronological Topics with Thematic Points, each citing [segment_ids]

Call 3: Decisions $\rightarrow$ Consensus Decisions (D_1, D_2) vs. Unresolved Items (*proposals, open questions, deferred*)

Call 4: Actions $\rightarrow$ Committed Tasks (T_1, T_2) with $\langle \text{Task, Owner, Deadline, [segment_ids]} \rangle$ vs. *Possible Tasks*

Call 5: Verification $\rightarrow$ Adversarial Cross-Check verifying candidates against cited context lines ± 2

$\rightarrow$ [Deterministic Grounding Engine] $\rightarrow$ [Final Meeting Record JSON]

Figure 1: The Five-Stage Sequential Documentation Workflow with Adversarial Verification.

7.1 Lexical Support Metric Formulation

Every proposed decision, task, and minute point must demonstrate verifiable textual overlap with the transcript segments it cites. The context window is constructed by expanding cited segment IDs by ± 1 line:

$$\mathcal{C}_{\text{context}} = \bigcup_{s \in \text{cited_ids}} \{s-1, s, s+1\} \cap \mathcal{S}_{\text{transcript}} \quad (10)$$

Text is normalized into content tokens by stripping standard English stopwords, removing punctuation, and applying a lightweight morphological stemmer:

$$\mathcal{T}(x) = \{\text{stem}(w) \mid w \in \text{words}(x), |w| \geq 3, w \notin \text{Stopwords}\} \quad (11)$$

The **Lexical Support Score** is defined as the fraction of item content tokens present within the expanded context tokens:

$$\text{Support}(\text{item}, \mathcal{C}_{\text{context}}) = \frac{|\mathcal{T}(\text{item}) \cap \mathcal{T}(\mathcal{C}_{\text{context}})|}{|\mathcal{T}(\text{item})|} \quad (12)$$

If $\text{Support} < 0.40$ (configurable via `config.yaml`), the item is rejected as unsupported, filtered from the public record, and logged to the `dropped` metadata register with the reason: **Insufficient lexical support in cited lines**.

7.2 Generic Pronoun Demotion and Owner Grounding

To eradicate phantom task ownership, proposed owner strings are scrutinized:

1. **Generic Pronoun Stripping:** Any owner string matching collective or indefinite terms (“i”, “me”, “we”, “us”, “our”, “team”, “everyone”, “everybody”, “someone”, “somebody”, “anyone”, “nobody”, “speaker”, “the team”, “engineering team”) is immediately stripped:

$$\text{owner} \leftarrow \text{None} \quad (\implies \text{Rendered as Unspecified}) \quad (13)$$

2. **Proximity Grounding Window:** If an owner name is proposed, that exact name string must appear within ± 4 segments of the cited transcript lines, or match an identified attendee in the meeting’s participant register. If the proposed owner does not appear in this window, the owner attribute is revoked ($\text{owner} \leftarrow \text{None}$) and recorded in `dropped`.

7.3 Verbatim Temporal Preservation

Deadlines are strictly validated against a ± 2 segment window. Content tokens of the deadline (e.g., “Thursday”, “fifteenth”) must appear verbatim within the cited dialogue. Furthermore, the system prohibits converting relative spoken deadlines into calendar timestamps: spoken expressions like “by Thursday” or “before the audit on the 15th” are preserved verbatim in their natural spoken form.

7.4 Semantic Deduplication and Canonical Ordering

If multiple topics or calls produce overlapping items, RapidFuzz token set ratio scoring identifies duplicates (≥ 88). When duplicates occur, the items are merged, preserving the maximal union of cited segment IDs. Canonical sequential identifiers are assigned: $D_1, D_2, \dots$ for decisions, $T_1, T_2, \dots$ for tasks, and $U_1, U_2, \dots$ for unresolved items.

8 Interactive Human-in-the-Loop Audio-Visual Workspace

The frontend (`frontend/src/`), built with Vite, React 19, TypeScript, and Zustand, embodies the *Evidence Follows You* philosophy. It rejects generic UI frameworks and Tailwind in favor of a bespoke, proofreading-inspired design system with CSS tokens (`--paper`, `--ink`, `--rule`, `--blue`, `--marker`).

8.1 Bidirectional Audio-Visual Seeking Architecture

The central interactive interface is the **Evidence Chip** (`frontend/src/components/record/EvidenceChip.tsx`). Every item in the left record pane displays an evidence badge indicating its source segment (e.g., ► 01:05 or ► 01:05 +1):

- **Hover Interaction:** Softly highlights the corresponding segments in the right-hand transcript pane via `focusStore`.
- **Click Interaction:** Computes the bounding temporal envelope $[t_{\text{start}}, t_{\text{end}}]$ of all cited segments. `WaveSurfer.js` creates a translucent yellow highlighter region (`--marker` at 35% opacity), seeks the audio playback head to $t_{\text{start}} - 1.0$ s (providing natural auditory context), plays through $t_{\text{end}} + 0.5$ s, and pauses automatically. Concurrently, the right pane smoothly scrolls the target segment into view with a pinned blue marker.

8.2 Interactive Correction Toggle and State Invalidation

In the Refined and Compare transcript views, applied corrections are rendered as underlined insertion marks (`<ins class="ins-mark">`). Clicking any edit opens the **Correction Popover** (`frontend/src/components/transcript/Correct`), displaying the struck-through original text, the replacement word, the safety justification, and an interactive toggle switch.

If a human reviewer toggles a correction from **Applied** to **Reverted**:

1. The client dispatches `PATCH /api/runs/{id}/corrections/{cid}`.
2. The backend updates `refined.json` and sets `record_stale = true` in run metadata.
3. The frontend immediately surfaces a prominent **Stale Banner** alerting the user that transcript modifications have invalidated the meeting record.
4. Clicking the “**Rewrite Record**” button initiates `POST /api/runs/{id}/rerun` with `from_stage="document"`, re-executing only the documentation stage in the background without re-transcribing the audio.

9 Export Subsystem and Structural Parity Verification

Implemented in `backend/verbatim/export/exporter.py`, Verbatim automatically generates eight distinct export deliverables packaged into a single download archive:

- `raw_transcript.txt` & `raw_transcript.srt`: Standard subtitle and timestamped text tracks.
- `raw_transcript.json` & `refined_transcript.json`: Machine-readable transcript schemas with sub-word timestamps and applied diff spans.
- `corrections.csv`: Complete audit spreadsheet containing $\langle \text{segment_id}, \text{time}, \text{original}, \text{corrected}, \text{reason}, \text{status} \rangle$.
- `meeting_record.json`: Complete structured schema of minutes, decisions, tasks, and dropped items.
- `meeting_record.md`: Formatted human-readable report rendered via Jinja2 template (`templates/meeting_record.md.j2`).
- `bundle.zip`: Comprehensive ZIP archive packaging all eight deliverables.

9.1 Automated Structural Parity Contract

A recurring defect in automated reporting systems is formatting drift: information present in a structured JSON export is omitted or distorted in the rendered Markdown document. Verbatim enforces an automated **Parity Verification Contract** (`verify_parity`):

$$\text{Markdown Table Rows} \equiv \text{JSON Action Items}, \quad \text{Markdown Bullets} \equiv \text{JSON Decisions} \quad (14)$$

The exporter parses decision bullet points and task table rows directly out of the rendered Markdown document using regular expressions and compares them against `meeting_record.json`. It asserts 1-to-1 equivalence in count, item IDs, and text content. The result is exposed via the API as `exportParity: {ok: true, decisions: N, tasks: M}`, providing reviewers with mathematical assurance of deliverable consistency.

10 Experimental Evaluation and Empirical Results

To evaluate Verbatim under realistic, adversarial conditions, the system was subjected to rigorous empirical testing against the canonical multi-speaker engineering deliberation (`samples/sample_meeting.mp3`, transcribed from `samples/meeting_script.md`).

10.1 Benchmark Scenario Description

The benchmark audio recording (1 minute 55 seconds, 16 kHz, featuring three distinct speakers: Maya, Priya, and Dan) was intentionally authored with subtle linguistic and technical edge cases designed to break standard meeting assistants:

- **Acoustic Traps:** Colloquial pronunciation of *Memcached* (“Memcatch”), *PostgreSQL* (“post-GIR SQL”), *OAuth scopes* (“OOFScores”), and *gRPC* (“GRPC”).
- **Decision Consensus vs. Deferred Deliberation:**

Table 4: Empirical Ground-Truth Verification Results on Canonical Benchmark (`eval/check_sample.py`).

Test Category	Evaluation Target	Observed Output / Classification	Status
Decisions Present	Redis session cache migration	<i>“Migrate session cache from Memcached to Redis”</i>	PASS (100%)
Decisions Present	PostgreSQL freeze (negative decision)	<i>“Do not upgrade PostgreSQL to version 16 this quarter”</i>	PASS (100%)
Decisions Absent	gRPC migration (deferred proposal)	Omitted from Decisions; classified into unresolved (deferred)	PASS (100%)
Decisions Absent	Grafana dashboards (tentative suggestion)	Omitted from Decisions; classified into unresolved (proposal)	PASS (100%)
Decisions Absent	Kafka alert discussion (informational)	Omitted from Decisions; noted in chronological minutes only	PASS (100%)
Unresolved Items	gRPC protocol migration	Present in unresolved with category deferred	PASS (100%)
Unresolved Items	Grafana dashboard inspection	Present in unresolved with category possible_task	PASS (100%)
Action Item 1	Redis Terraform module	Owner: Dan Deadline: “by Thursday” Cited: [10, 11]	PASS (100%)
Action Item 2	OAuth scopes documentation	Owner: Priya Deadline: Unspecified (null) Cited: [20, 21]	PASS (100%)
Action Item 3	Database credentials rotation	Owner: Unspecified (null) Deadline: “before the audit / 15th”	PASS (100%)
Forbidden Tasks	Grafana dashboard inspection	Correctly excluded from committed action_items table	PASS (100%)
Export Parity	Markdown vs JSON Consistency	Strict 1-to-1 match across all decisions ($N = 2$) and tasks ($M = 3$)	PASS (100%)

- Consensus Decision 1: Migrate session cache from Memcached to Redis.
- Consensus Decision 2 (Negative Decision): Do not upgrade PostgreSQL to version 16 this quarter.
- Deferred Proposal (Non-Decision): Moving internal services to gRPC (discussed, but explicitly deferred by Maya to next sprint).

• **Task Allocation and Attribution Traps:**

- Task 1 (Committed): Dan assigned to write Terraform module for Redis, with deadline *“by Thursday”*.
- Task 2 (Committed, Unspecified Deadline): Priya assigned to update OAuth scopes documentation; deadline explicitly stated as unknown (*“No date yet”*).
- Task 3 (Committed, Unspecified Owner): Rotate staging database credentials before the audit on the 15th; owner left unassigned (*“Someone has to rotate...I’ll leave that open”*).
- Non-Task / Speculation: Maya suggests Dan look at Grafana dashboards; Dan replies *“We’ll see”* (tentative idea, forbidden from appearing in action items).

10.2 Ground-Truth Benchmark Results

The benchmark execution was evaluated using the automated evaluation harness (`eval/check_sample.py`) against ground-truth specifications (`samples/expected.json`). The results are presented in Table 4.

10.3 Word Error Rate (WER) and Technical Vocabulary Recovery

Acoustic transcription across the benchmark recording generated 45 raw segments. Faster-Whisper initially transcribed specialized engineering terms with phonetic misspellings:

- Segment 4: *“we’re still on Memcatch”* → Refined: *“we’re still on Memcached”*
- Segment 17: *“On post-GIR SQL, we are not going to upgrade”* → Refined: *“On PostgreSQL, we are not going to upgrade”*
- Segment 20: *“update the OOFscopes documentation”* → Refined: *“update the OAuth scopes documentation”*
- Segment 13: *“move all internal services from REST to GRPC”* → Refined: *“move all internal services from REST to gRPC”*

The Refiner successfully proposed corrections for all four technical terms. All four proposals passed the seven safety guardrails: number invariance (Rule 4) verified that numbers (340 ms, 250 ms, 90 ms, 5%, 16, 15th, \$12,000) were unmodified, and negation invariance (Rule 5) verified that critical negative constraints (*“not going to upgrade”*, *“must not exceed”*) were strictly preserved.

10.4 Latency and Resource Consumption Across Hardware Profiles

Table 5 summarizes empirical execution latencies and peak memory footprints measured across hardware configurations.

Table 5: Stage-by-Stage Latency and VRAM Consumption (1m 55s Meeting Audio).

Pipeline Stage	Standard (RTX 4060)	Lite (Apple Silicon)	Lite (x86 CPU)
1. Audio Ingest	0.42 s	0.58 s	0.65 s
2. STT (Whisper)	4.12 s (large-v3, fp16)	8.35 s (distil, int8)	11.20 s (distil, int8)
3. Refinement	6.85 s (qwen3:8b)	5.10 s (qwen3:4b)	14.30 s (qwen3:4b)
4. Documentation	11.20 s (gemma3:12b)	7.45 s (gemma3:4b)	22.80 s (gemma3:4b)
5. Export & Parity	0.18 s	0.22 s	0.25 s
Total Runtime	22.77 s	21.70 s	49.20 s
Peak VRAM	5.82 GB	Unified Memory	0 MB (RAM: 3.2 GB)

10.5 Regression Test Suite Breakdown

The entire codebase maintains an automated test suite comprising 80 automated tests spanning unit, integration, and end-to-end specifications. Table 6 details the test distribution and status.

Table 6: Automated Test Suite Distribution and Verification Status.

Subsystem Test Suite	Target Verification Scope	Passing Tests
test_ingest.py	Ingest validation, PyAV decoding, silence dBFS, peaks	6 / 6 passed
test_stt.py	Prompt construction, VAD, repetition collapse, no-speech	4 / 4 passed
test_hints.py	RapidFuzz acoustic hints and domain vocabulary scoring	5 / 5 passed
test_guardrails.py	The 7 deterministic safety guardrails (numbers, negations)	9 / 9 passed
test_apply.py	Non-overlapping span splicing, propagation, case-fallback	7 / 7 passed
test_grounding.py	Lexical support scoring, owner demotion, deadline checks	9 / 9 passed
test_document.py	5-stage secretary orchestration, schema assembly, events	1 / 1 passed
test_exporter.py	SRT/TXT generation, CSV audit, Jinja2 Markdown parity	5 / 5 passed
test_api.py	REST routes, HTTP 206 Range audio, SSE event replay	6 / 6 passed
test_phase7_e2e.py	End-to-end pipeline streaming with SPA serving	2 / 2 passed
test_phase9_resilience.py	Ollama mid-run disconnect, CUDA fallback, export parity	8 / 8 passed
frontend (vitest)	Zustand reducers, WaveSurfer playback, Record/Transcript UI	28 / 28 passed
Total Automated Tests	Complete End-to-End System Verification	80 / 80 passed (100%)

11 Discussion, Trade-Offs, and Limitations

The engineering choices embodied in Verbatim prioritize *factual veracity and auditable accountability* over unconstrained generative fluency. In this section, we analyze the architectural trade-offs, edge-case failure modes, and operational boundaries of the system.

11.1 Trade-Off: Deterministic Rigidity vs. Generative Fluency

The decision to enforce hard multiset invariance (e.g., Rule 4: Number Invariance and Rule 5: Negation Invariance) occasionally rejects semantically harmless rephrasings. For example, if a speaker says “*not inexpensive*” and the Refiner proposes “*costly*”, the guardrail engine detects that the negation token “*not*” was dropped and rejects the edit.

While this constitutes a false positive rejection from a pure stylistic perspective, we argue that in mission-critical engineering secretary applications, **soundness must supersede fluency**. Allowing an LLM to alter negation tokens or numeric quantities introduces unacceptable risk of silent inversion of meaning (e.g., converting “*we must not deploy*” to “*we must deploy*”). By strictly logging rejected proposals to **corrections.csv** and rendering them in the interactive UI comparison view, human reviewers retain complete visibility and final agency.

11.2 Acoustic Boundary Conditions: Cross-Talk and Audio Overlap

Like all single-channel speech recognition systems, Faster-Whisper exhibits degraded performance when multiple participants speak simultaneously (cross-talk). In overlapping speech regimes, Whisper tends to transcribe the dominant acoustic stream while dropping background utterances, or interleaving phonemes into ungrammatical fragments.

While Verbatim’s lexical support scoring successfully identifies and drops hallucinated summary claims arising from garbled cross-talk segments, complete recovery of overlapping speakers requires multi-channel microphone arrays or specialized neural speaker separation frontends (e.g., Conformer-based source separation).

11.3 Scalability to Multi-Hour Deliberations

For meetings exceeding two hours in duration, the flat sliding-window documentation approach approaches the boundary of single-pass context windows. While Verbatim addresses this via duration caps per hardware profile (e.g., 60 minutes for **standard**), scaling to multi-hour symposiums will require a hierarchical Map-Reduce summarization topology: partitioning the meeting into distinct agenda chapters, extracting local decisions and tasks per chapter, and performing an overarching global reconciliation pass.

12 Conclusion

In this technical report, we presented **Verbatim**, a robust, hallucination-resistant meeting assistant that bridges the gap between raw audio recordings and actionable engineering records. By rejecting the naive single-prompt paradigm, Verbatim establishes a new architectural benchmark defined by:

1. **Bidirectional Audio-Visual Grounding:** Eliminating the trust deficit by binding every synthesized decision, task, and minute point directly to underlying audio ranges and waveform visualizers with single-click verification.
2. **Deterministic Guardrail Enforcement:** Applying seven mathematical and lexical invariance checks to govern probabilistic LLM refiners, preventing hallucinated revisions, numeric alterations, or polarity inversions.
3. **Decomposed Secretary Pipeline:** Isolating consensus decisions from tentative debates, demoting generic pronouns to explicit **Unspecified** states, and verifying claims through localized lexical support scoring (≥ 0.40).
4. **Local-First Operational Integrity:** Delivering responsive SSE streaming, zero-spinner progress rendering, hardware-adaptive profile execution, and automated Markdown-to-JSON structural parity assurance.

Empirical evaluation on complex multi-speaker technical deliberations confirms a 100% ground-truth verification pass rate across all decision and action item constraints, demonstrating that meeting intelligence systems can achieve both high generative utility and rigorous mathematical accountability.

References

- [1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever. Robust speech recognition via large-scale weak supervision. In *International Conference on Machine Learning (ICML)*, pages 28492–28518, 2023.
- [2] G. Klein, D. Hernandez, and OpenNMT Team. *CTranslate2: Fast Inference Engine for Transformer Models*. GitHub repository: <https://github.com/OpenNMT/CTranslate2>, 2020.
- [3] Qwen Team. Qwen2.5: A foundation language model with comprehensive capabilities. *arXiv preprint arXiv:2409.12191*, 2024.
- [4] Gemma Team, Google DeepMind. Gemma 3: Open models for multimodal and multilingual reasoning. *Technical Report*, Google DeepMind, 2025.
- [5] M. Bachmann. *RapidFuzz: Rapid fuzzy string matching in Python and C++*. GitHub repository: <https://github.com/rapidfuzz/RapidFuzz>, 2021.
- [6] Silero Team. *Silero VAD: Pre-trained Enterprise-grade Voice Activity Detector*. GitHub repository: <https://github.com/snakers4/silero-vad>, 2021.
- [7] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.
- [8] J. Carletta. Unleashing the killer corpus: experiences in creating the AMI meeting corpus. *Language Resources and Evaluation*, 41(2):181–190, 2007.
- [9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 33:9459–9474, 2020.